ЧАСТЬ 2. МАССИВЫ, СТРОКИ, МЕТОДЫ • ЗАНЯТИЕ 1 ИЗ 2

2.1. Массивы и список List

К концу занятия ты сможешь:

- хранить много значений под одним именем — в массиве;
- брать ячейку по её номеру и перебирать все ячейки циклом;
- находить сумму, среднее, самое большое и самое маленькое значение.

Разминка

Вопрос 1

Что выведет программа?

Program.cs

```
for (int i = 1; i <= 3; i++)
{
    Console.WriteLine(i * 10);
}
```

Вопрос 2

Чем цикл do while отличается от while?

Вопрос 3

Где-то в цикле переменная получает неправильное значение. Как найти этот шаг, не меняя код наугад?

ЗАДАЧА, КОТОРУЮ ПОКА НЕЧЕМ РЕШИТЬ

Турнирная таблица на 10 игроков

Program.cs

```
int score1 = 120;
int score2 = 95;
int score3 = 140;
int score4 = 80;
// и так до score10

int best = score1;
if (score2 > best) best = score2;
if (score3 > best) best = score3;
if (score4 > best) best = score4;
// и ещё 6 таких строк
```

А если игроков 1000?

1000 переменных и 999 строк с if. Перепутать score7 и score8 в одной из них — проще простого.

Сегодня научимся находить лидера среди любого числа игроков — за 5 строк кода.

ПЛАН

План занятия

УРОК 1 • 40 МИНУТ

- Разминка и план
- Блок 1. Массив: создание, индексы, Length
- Перебор циклами for и foreach
- Живой кодинг «Плейлист»
- Вопросы и практика «Расписание уроков»

УРОК 2 • 40 МИНУТ

- Блок 2. Сумма и среднее
- Максимум, минимум и номер лучшего
- Подсчёт по условию
- Практика «Шагомер за неделю»
- Мини-проект «Дартс» и итоги

БЛОК 1 ИЗ 2 • УРОК 1

Массив: много значений под одним именем

В этом блоке ты научишься:

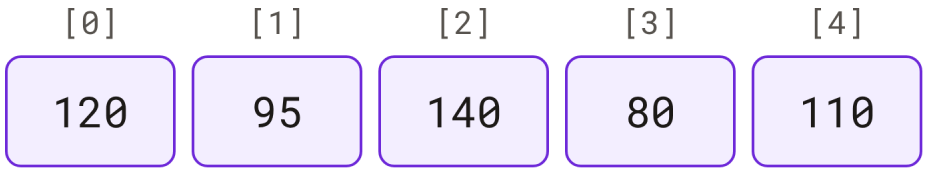
- создавать массив и заполнять его значениями;
- брать и менять значение ячейки по её номеру;
- перебирать все ячейки циклами for и foreach.

Массив — ряд пронумерованных ячеек

Массив — это ряд ячеек одного типа под одним общим именем. У каждой ячейки есть номер.

Как шкафчики в раздевалке: один ряд, у каждого шкафчика номер, в каждом лежит одна вещь.

Массив scores:



Было: 5 переменных

Program.cs

```
int score1 = 120;
int score2 = 95;
int score3 = 140;
int score4 = 80;
int score5 = 110;
```

Стало: один массив

Program.cs

```
int[] scores = { 120, 95, 140, 80, 110 };
```

Правило: много значений одного типа — один массив, а не много переменных.

Создание массива: new

Program.cs

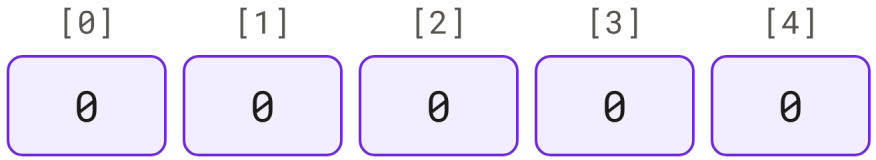
```
int[] scores = new int[5];
```

int[] — тип: массив целых чисел

scores — имя массива

new int[5] — создать 5 ячеек для чисел

Что лежит в scores сразу после создания:



Новый массив заполнен значениями по умолчанию: у int и double — 0, у bool — false, у string ячейки пустые.

Ошибка: new int[] без размера. Visual Studio пишет CS1586: Array creation must have array size or array initializer (нужен размер или список значений).

Правило: размер задаётся при создании и потом не меняется.

Массив сразу со значениями: { }

Program.cs

```
int[] marks = { 5, 4, 3, 5, 4 };
string[] songs = { "Believer", "Numb", "Thunder" };
double[] prices = { 99.5, 150, 49.9 };
```

индекс	marks	songs	prices
0	5	Believer	99,5
1	4	Numb	150
2	3	Thunder	49,9
3	5	нет ячейки	нет ячейки
4	4	нет ячейки	нет ячейки

Размер считается сам: в marks 5 ячеек, в songs и prices по 3.

В коде дробные пишутся через точку: 99.5. Русская Windows покажет на экране 99,5.

Ошибка: int[] marks = { 5, "четыре" }; — CS0029: Cannot implicitly convert type 'string' to 'int'. Все значения в массиве одного типа.

Правило: значения известны заранее — перечисли их в фигурных скобках.

Индекс: номер ячейки, счёт с нуля

Program.cs

```
string[] songs = { "Believer", "Numb", "Thunder" };
Console.WriteLine(songs[0]); // первая песня
songs[2] = "Radioactive"; // заменили третью
Console.WriteLine(songs[2]);
```

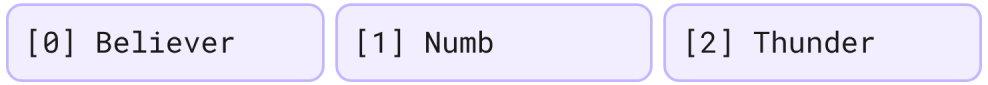
Вывод в консоли

```
Believer
Radioactive
```

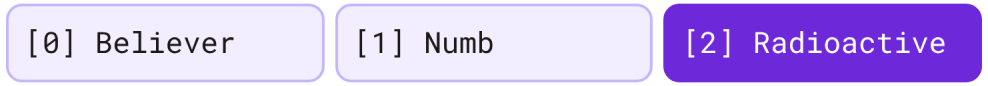
Индекс — номер ячейки в квадратных скобках. Первая ячейка — [0], вторая — [1].

songs[2] можно и прочитать, и записать, как обычную переменную.

До:



После songs[2] = "Radioactive":



Правило: у N-го элемента индекс N – 1. Первый — [0].

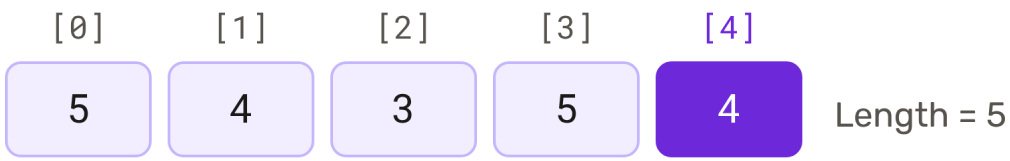
Length: сколько ячеек в массиве

Program.cs

```
int[] marks = { 5, 4, 3, 5, 4 };
Console.WriteLine(marks.Length); // сколько ячеек
Console.WriteLine(marks[marks.Length - 1]); // последняя
```

Вывод в консоли

```
5
4
```



Правило: индексы идут от 0 до Length – 1.

Length — число ячеек массива. Пишется через точку после имени, без круглых скобок.

5 ячеек — индексы от 0 до 4. Последний индекс на 1 меньше длины.

Зачем: добавят шестую оценку — код с Length менять не придётся, а код с числом 4 станет неверным.

БЛОК 1 • ЛОВУШКА

Ловушка: выход за границу массива

Program.cs

```
int[] marks = { 5, 4, 3, 5, 4 };
Console.WriteLine(marks[5]); // шестая оценка?
```

Вывод в консоли

```
Unhandled exception. System.IndexOutOfRangeException: Index was outside the bounds of the array.
   at Program.<Main>$(String[] args) in C:\Projects\ArraysDemo\Program.cs:line 2
```

Перевод: необработанное исключение — индекс вышел за границы массива. Ошибка в строке 2 файла Program.cs.

Почему

5 ячеек — индексы 0–4. Ячейки [5] нет. Компилятор этого не видит, программа падает во время работы.

Как читать

IndexOutOfRangeException — индекс вне границ. line 2 — номер строки, где это случилось.

Как правильно

Последняя ячейка — marks[marks.Length - 1]. Индекс всегда меньше Length.

Перебор массива циклом for

Program.cs

```
string[] songs = { "Believer", "Numb", "Thunder" };
for (int i = 0; i < songs.Length; i++)
{
    Console.WriteLine($"{i + 1}. {songs[i]}");
}
```

Вывод в консоли

```
1. Believer
2. Numb
3. Thunder
```

Шаг	i	i < Length?	Вывод
1	0	0 < 3, да	1. Believer
2	1	1 < 3, да	2. Numb
3	2	2 < 3, да	3. Thunder
4	3	3 < 3, нет	цикл закончен

Зачем: одна строка вывода вместо трёх — и для 3, и для 1000 песен.

Правило: i идёт от 0, пока i < Length — так цикл проходит каждую ячейку один раз.

Перебор циклом foreach

foreach — «для каждого»: по очереди берёт каждый элемент массива и кладёт его в переменную. Как учитель, который идёт вдоль ряда и здоровается с каждым.

Program.cs

```
string[] friends = { "Аня", "Макс", "Лиза" };
foreach (string friend in friends)
{
    Console.WriteLine("Привет, " + friend + "!");
}
```

Вывод в консоли

```
Привет, Аня!
Привет, Макс!
Привет, Лиза!
```

Чего foreach не умеет: менять ячейки

Program.cs

```
int[] marks = { 5, 4, 3 };
foreach (int mark in marks)
{
    mark = 5; // хотим исправить оценку
}
```

Ошибка при сборке

```
error CS1656: Cannot assign to 'mark' because it is a
'foreach iteration variable'
```

Перевод: нельзя присвоить значение mark — это переменная цикла foreach.

Правило: только прочитать каждый — foreach. Нужен номер или нужно менять ячейки — for.

Живой кодинг: «Плейлист»

1. Массив songs из 3 названий песен.
2. Массив minutes на столько же ячеек: new int[songs.Length].
3. Цикл for: спросить длительность каждой песни и записать в minutes[i].
4. Второй цикл: вывести «1. Believer: 4 мин» и так далее.
5. Здесь будет ошибка. Читаем сообщение и ищем её отладчиком.
6. Исправляем, запускаем, сохраняем в Git.

Место для скриншота: окно исключения в Visual Studio и окно Locals со значением i = 3

Вывод в консоли (в конце)

```
Сколько минут длится Believer?
4   <- ввёл пользователь
Сколько минут длится Numb?
3   <- ввёл пользователь
Сколько минут длится Thunder?
3   <- ввёл пользователь
Мой плейлист:
1. Believer: 4 мин
2. Numb: 3 мин
3. Thunder: 3 мин
```

Стрелка — пометка для тебя, в программе её нет.

Вопросы по блоку 1

Вопрос 1

Что выведет программа? Почему не 5?

```
int[] marks = { 5, 4, 3, 5, 4 };
Console.WriteLine(marks[1]);
```

Вопрос 2

Что случится, если в цикле написать так? На каком шаге?

```
for (int i = 0; i <= marks.Length; i++)
{
    Console.WriteLine(marks[i]);
}
```

Вопрос 3

Нужно поставить всем оценкам 5. Почему через foreach не получится, а через for получится?

Задание: «Расписание уроков»

- Программа спрашивает названия 5 уроков на завтра и сохраняет их в массив.
- Потом выводит расписание: «1 урок: Алгебра» и так далее.
- В конце отдельно пишет первый и последний урок.

Подсказка

Номер урока при выводе — это i + 1. Последний урок найди через Length, а не числом 4.

★ **Успел раньше?** Спроси номер урока и покажи его название. Номер 0 или 7 — пиши «Такого урока нет», программа не падает.

Застрял — 15 минут ищешь сам, потом спрашиваешь: что делал, что ожидал, что получил.

Что должно получиться

Вывод в консоли

```
Урок 1:
Алгебра   <- ввёл пользователь
Урок 2:
История   <- ввёл пользователь
Урок 3:
Физкультура   <- ввёл пользователь
Урок 4:
Английский   <- ввёл пользователь
Урок 5:
Информатика   <- ввёл пользователь
Расписание на завтра:
1 урок: Алгебра
2 урок: История
3 урок: Физкультура
4 урок: Английский
5 урок: Информатика
Первый урок: Алгебра
Последний урок: Информатика
```

Стрелка — пометка для тебя, в программе её нет.

Проверь себя

- Введи другие предметы — расписание тоже меняется.
- Последний урок найден через Length: если сделать массив на 6 уроков, ответ всё равно верный.
- Программа не падает с IndexOutOfRangeException.

★★ «Код сейфа»

В программе записан код сейфа:

```
int[] code = { 7, 2, 9, 4 };
```

Игрок вводит 4 цифры по одной. Программа говорит, сколько цифр стоят на своих местах, и открыт ли сейф.

Подумай: как сделать, чтобы игрок пробовал снова, пока не откроет сейф, а в конце узнал число попыток?

Вывод в консоли

```
Цифра 1:
7   <- ввёл пользователь
Цифра 2:
1   <- ввёл пользователь
Цифра 3:
9   <- ввёл пользователь
Цифра 4:
3   <- ввёл пользователь
На своих местах: 2 из 4
Сейф закрыт
```

Стрелка — пометка для тебя, в программе её нет.

БЛОК 2 ИЗ 2 • УРОК 2

Считаем по массиву

В этом блоке ты научишься:

- складывать все элементы и находить среднее;
- находить самое большое и самое маленькое значение и его номер;
- считать элементы, которые подходят под условие.

Сумма: переменная-накопитель

Накопитель — переменная, в которую по очереди добавляют каждый элемент. Как копилка: бросаешь монеты по одной, в конце считаешь.

Program.cs

```
int[] goals = { 2, 0, 3, 1 }; // голы за 4 матча
int sum = 0; // копилка пуста
foreach (int goal in goals)
{
    sum += goal; // бросили в копилку
}
Console.WriteLine($"Всего голов: {sum}");
```

Вывод в консоли

```
Всего голов: 6
```

Шаг	goal	sum до	sum после
1	2	0	2
2	0	2	2
3	3	2	5
4	1	5	6

Ошибка: int sum = 0; внутри цикла. Копилка обнуляется на каждом шаге, а после цикла sum не видна: CS0103: The name 'sum' does not exist in the current context (имени sum здесь не существует).

Правило: накопитель объявляют до цикла и начинают с 0.

БЛОК 2 • ЛОВУШКА

Среднее: ловушка целочисленного деления

Program.cs

```
int[] marks = { 5, 5, 4 };
int sum = 0;
foreach (int mark in marks)
{
    sum += mark;
}
Console.WriteLine(sum / marks.Length); // 14 / 3
Console.WriteLine((double)sum / marks.Length);
double average = (double)sum / marks.Length;
Console.WriteLine($"{average:F2}");
```

Вывод в консоли

```
4
4,666666666666667
4,67
```

Правило: среднее = (double)сумма / Length.

Ошибка: 4

sum и Length — оба int, поэтому деление целочисленное: дробная часть отбрасывается (тема 1.3).

Правильно: 4,67

(double) перед sum делает деление дробным. {average:F2} оставляет 2 знака после запятой.

Максимум: «действующий чемпион»

Первый элемент — чемпион. Каждый следующий сравниваем с ним: кто больше, тот новый чемпион. Пример: прыжки в длину на физкультуре, в сантиметрах.

Program.cs

```
int[] jumps = { 410, 455, 430, 470, 445 };
int best = jumps[0]; // чемпион — первый прыжок
for (int i = 1; i < jumps.Length; i++)
{
    if (jumps[i] > best)
    {
        best = jumps[i]; // новый чемпион
    }
}
Console.WriteLine($"Лучший прыжок: {best} см");
```

Вывод в консоли

```
Лучший прыжок: 470 см
```

Старт: best = 410

i	jumps[i]	больше best?	best
1	455	455 > 410, да	455
2	430	430 > 455, нет	455
3	470	470 > 455, да	470
4	445	445 > 470, нет	470

Правило: цикл начинается с i = 1 — элемент [0] уже чемпион.

Номер лучшего: храним индекс

Часто важно не только значение, но и где оно: какая попытка, какой день. Храним номер чемпиона, а значение берём по номеру: jumps[bestIndex].

Program.cs

```
int[] jumps = { 410, 455, 430, 470, 445 };
int bestIndex = 0; // номер лучшей попытки
for (int i = 1; i < jumps.Length; i++)
{
    if (jumps[i] > jumps[bestIndex])
    {
        bestIndex = i;
    }
}
Console.WriteLine($"Попытка {bestIndex + 1}: {jumps[bestIndex]} см");
```

Вывод в консоли

```
Попытка 4: 470 см
```

Почему + 1?

Индексы с 0, а попытки люди считают с 1.

Два равных рекорда?

Знак > не меняет чемпиона на равного: останется первый. С >= останется последний.

Правило: нужен номер — храни индекс, значение всегда возьмёшь по нему.

Минимум и ловушка «чемпиона-нуля»

Минимум ищут так же, только со знаком <. А вот частая ошибка при поиске максимума зимой:

Ошибка

Program.cs

```
int[] temps = { -3, -8, -1, -5 };
int max = 0; // чемпион из воздуха
foreach (int temp in temps)
{
    if (temp > max)
    {
        max = temp;
    }
}
Console.WriteLine($"Теплее всего: {max}");
```

```
Теплее всего: 0
```

Правильно

Program.cs

```
int[] temps = { -3, -8, -1, -5 };
int max = temps[0]; // чемпион из массива
foreach (int temp in temps)
{
    if (temp > max)
    {
        max = temp;
    }
}
Console.WriteLine($"Теплее всего: {max}");
```

```
Теплее всего: -1
```

Ни один день не был теплее 0, и «чемпион-ноль» так и не проиграл. Но температуры 0 в массиве нет вообще.

Правило: чемпиона берут из массива — temps[0], а не 0. Для минимума: if (temp < min).

Подсчёт по условию: счётчик

Счётчик — переменная, к которой прибавляют 1 каждый раз, когда элемент подходит под условие. Накопитель прибавляет значение элемента, счётчик — единицу.

Program.cs

```
int[] marks = { 5, 4, 5, 3, 5, 4 };
int fives = 0; // счётчик пятёрок
foreach (int mark in marks)
{
    if (mark == 5)
    {
        fives++;
    }
}
Console.WriteLine($"Пятёрок: {fives}
из {marks.Length}");
```

Вывод в консоли

```
Пятёрок: 3 из 6
```

mark	mark == 5?	fives
5	да	1
4	нет	1
5	да	2
3	нет	2
5	да	3
4	нет	3

Правило: счётчик — 0 до цикла и ++ внутри if.

Вопросы по блоку 2

Вопрос 1

Почему при поиске максимума начинаем с jumps[0], а не с 0? Когда 0 даст неверный ответ?

Вопрос 2

Оценки 5, 5, 4. Что выведет Console.WriteLine(sum / marks.Length); и почему не 4,67?

Вопрос 3

Чем счётчик отличается от накопителя? Придумай по примеру из своего телефона или игры.

Задание: «Шагомер за неделю»

- Программа спрашивает, сколько шагов ты прошёл в каждый из 7 дней.
- Выводит, сколько всего шагов и сколько в среднем за день (2 знака после запятой).
- Выводит номер лучшего дня и сколько дней выполнена норма: 8000 шагов и больше.

Подсказка

Сумма — накопитель, норма — счётчик, лучший день — храни индекс. Для среднего не забудь (double).

★ **Успел раньше?** Выводи не номер дня, а его название: заведи массив days = { "Пн", "Вт", "Ср", "Чт", "Пт", "Сб", "Вс" }.

Что должно получиться

Вывод в консоли

```
День 1:
6500   <- ввёл пользователь
День 2:
9200   <- ввёл пользователь
День 3:
8000   <- ввёл пользователь
День 4:
4300   <- ввёл пользователь
День 5:
10150   <- ввёл пользователь
День 6:
7600   <- ввёл пользователь
День 7:
8800   <- ввёл пользователь
Всего шагов: 54550
В среднем за день: 7792,86
Лучший день: 5
Дней с нормой: 4
```

Стрелка — пометка для тебя, в программе её нет.

Проверь себя

- День ровно с 8000 шагов засчитан в норму.
- Среднее с дробной частью: 7792,86, а не 7792.
- Если все дни одинаковые, лучший день — 1.

★★ «Серебряный призёр»

Очки игроков турнира:

```
int[] scores = { 120, 95, 140, 80, 140, 110 };
```

Найди первое и второе место. Второе место — самое большое число, которое меньше первого. Два раза по 140 — это одно первое место.

Условие: один проход цикла, без сортировки.

Вывод в консоли

```
Первое место: 140
Второе место: 120
```

Вопросы по занятию

Вопрос 1

Оценки 5, 4, 3. Сумму посчитали так. Что изменится в результате?

```
for (int i = 0; i < marks.Length - 1; i++)
{
    sum += marks[i];
}
```

Вопрос 2

Для суммы удобен foreach, а для номера лучшего нужен for. Почему?

Вопрос 3

Массив создан как new int[0], а поиск максимума начинается с int best = scores[0]; — что случится? Вспомни ловушку блока 1.

Мини-проект: «Дартс»

Тренажёр дартс. Ты решаешь, сколько бросков сделать, а компьютер «бросает» дротики: каждый бросок — от 0 до 10 очков.

Программа показывает все броски и считает статистику: сумму, средний бросок, лучший бросок и попадания в яблочко.

Это пригодится в Проекте 2 «Игра на поле»: поле тоже хранится в массиве, а счёт и рекорд считаются так же.

Вывод в консоли

```
Сколько бросков сделать? (от 1 до 20)
25   <- ввёл пользователь
Нужно число от 1 до 20
Сколько бросков сделать? (от 1 до 20)
5   <- ввёл пользователь
Бросок 1: 7
Бросок 2: 10
Бросок 3: 3
Бросок 4: 10
Бросок 5: 6
Сумма: 36
Средний бросок: 7,20
Лучший бросок: 10 (бросок 2)
В яблочко: 2
```

Стрелка — пометка для тебя, в программе её нет. Броски случайные: у тебя будут другие числа.

Что должна уметь программа

1

Спрашивает, сколько бросков сделать, и переспрашивает, если ввели буквы, 0 или больше 20.

2

Показывает каждый бросок с его номером.

3

Считает сумму очков и средний бросок с двумя знаками после запятой.

4

Находит лучший бросок с номером и считает попадания в яблочко — броски на 10 очков.

План по шагам

Шаг	Что сделать	Тема	Как проверить
1	Спросить число бросков через int.TryParse и переспрашивать в цикле do while	1.2, 1.5	Введи «abc» и 0 — программа пишет «Нужно число от 1 до 20» и спрашивает снова
2	Создать массив darts на count ячеек, заполнить случайными числами от 0 до 10, вывести броски	1.5, 2.1	Запусти — видишь «Бросок 1: …» столько раз, сколько ввёл
3	Посчитать сумму и средний бросок	1.3, 2.1	Сложи броски в уме — сумма совпадает; среднее с запятой, например 7,20
4	Найти лучший бросок с номером и число попаданий в яблочко	1.4, 2.1	Если 10 выпало дважды — в яблочко 2, номер лучшего — первая десятка

Заготовка

Program.cs

```
// Шаг 1. Спрашиваем, пока не введут число от 1 до 20
int count;
bool isNumber;
do
{
    Console.WriteLine("Сколько бросков сделать? (от 1 до 20)");
    isNumber = int.TryParse(Console.ReadLine(), out count);
    // если число не подходит, напиши «Нужно число от 1 до 20»
} while (!isNumber || count < 1 || count > 20);

// Шаг 2. Массив darts на count ячеек, броски random.Next(0, 11)
// Шаг 3. Сумма — накопитель, среднее — не забудь (double)
// Шаг 4. Лучший — храни индекс, «в яблочко» — счётчик десяток
```

Запись out count

Та же, что out int number из темы 1.2, только переменная count объявлена заранее.

Random

Создай Random random = new Random(); один раз, до цикла. Next(0, 11) даёт число от 0 до 10: верхняя граница не входит (тема 1.5).

Готово, если…

- Если ввести «abc» или 0, программа пишет «Нужно число от 1 до 20» и спрашивает снова.
- При 1 броске программа не падает, а лучший бросок — он сам, бросок 1.
- Сумма совпадает с подсчётом в уме, среднее выводится с двумя знаками: например, 7,20.
- Если 10 выпало дважды, «В яблочко: 2», а лучший бросок — первая десятка.

★ Дуэль

Компьютер бросает столько же раз за соперника. В конце: «Игрок: 36, соперник: 31. Победил игрок».

★★ Серия

Найди самую длинную серию бросков от 8 очков подряд: «Лучшая серия: 3 броска подряд».

Каждую строку своего кода ты должен уметь объяснить. Готово — сохрани в Git и отправь на GitHub.

Челлендж: «Погода за месяц»

Программа придумывает погоду на 30 дней: случайная температура от –10 до 10 градусов. Затем она:

- выводит температуру каждого дня;
- находит самый тёплый день: его номер и температуру;
- считает, сколько дней было ниже нуля;
- находит самую длинную серию морозных дней подряд;
- рисует для каждого дня «столбик»: столько символов #, сколько градусов тепла, или столько *, сколько градусов мороза.

Итоги занятия

Сегодня я научился:

- создавать массив и заполнять его — сразу или в цикле;
- брать ячейку по индексу и не выходить за границу;
- перебирать массив через for и foreach и выбирать, какой цикл нужен;
- находить сумму, среднее, лучший и худший результат с его номером;
- считать, сколько элементов подходит под условие.

В следующий раз:

индексы с конца и диапазоны, полезные методы Array и список List — массив, который умеет расти.